UNIVERSIDAD TECNOLOGICA DE TEHUACAN

PROGRAMA EDUCATIVO EN:

Tecnologías de la Información en Área Desarrollo
de Software Multiplataforma

NOMBRE DEL PROFESOR:

JOSÉ MIGUEL CARRERA PACHECO

INTEGRANTES DEL EQUIPO:

Ciro Julian Cervantes Zamora

MATERIA:

Desarrollo Web Profesional

TEMA:

Autenticación y verificación

TEHUACÁN, PUEBLA, ENERO-ABRIL 2026

1. Autenticación vs Autorización

Dos de los conceptos fundamentales en seguridad de software son la autenticación y la autorización. Aunque a menudo se usan de manera intercambiable en el lenguaje cotidiano, representan procesos distintos con propósitos muy diferentes.

1.1 ¿Que es la Autenticación?

La autenticación es el proceso mediante el cual un sistema verifica la identidad de un usuario o entidad. En términos simples, responde a la pregunta: ¿Quién eres tú? El sistema valida que el usuario es quien dice ser antes de permitir cualquier acceso.

Mecanismos comunes de autenticación:

- Contraseñas: Factor de conocimiento (algo que sabes).
- Biometría: Huella dactilar, reconocimiento facial (algo que eres).
- Tokens físicos o llaves de seguridad (algo que tienes).
- Autenticación multifactorial (MFA): Combinación de dos o más de los anteriores.

1.2 ¿Que es la Autorización?

La autorización es el proceso que determina que acciones o recursos puede acceder un usuario ya autenticado. Responde a la pregunta: ¿Que puedes hacer? Incluso si el sistema sabe quién eres, no necesariamente tienes acceso a todo.

Ejemplo práctico: Un empleado puede autenticarse en el sistema de su empresa (autenticación exitosa), pero solo puede ver los documentos de su departamento y no los de Recursos Humanos (autorización limitada).

1.3 Tabla Comparativa

Autenticación	Autorización
Verifica identidad del usuario	Determina permisos del usuario
Responde: ¿Quién eres?	Responde: ¿Que puedes hacer?
Ocurre primero en el flujo	Ocurre después de autenticar
Ejemplo: Login con contraseña	Ejemplo: Solo admins pueden borrar
JWT, OAuth, Sesiones	RBAC, ABAC, ACL, Scopes

2. JWT, Sesiones y OAuth

2.1 JSON Web Tokens (JWT)

Un JWT (JSON Web Token) es un estándar abierto (RFC 7519) que define una forma compacta y autosuficiente de transmitir información entre partes como un objeto JSON firmado digitalmente. Es especialmente popular en APIs REST y arquitecturas sin estado (stateless).

Estructura de un JWT: Se compone de tres partes separadas por puntos (.):

- Header: Especifica el tipo de token y el algoritmo de firma (ej. HS256, RS256).
- Payload: Contiene los 'claims' o afirmaciones, como el ID del usuario, roles y tiempo de expiracion.
- Signature: Firma criptografica que garantiza que el token no fue alterado.

Ejemplo:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiIxMjMiLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE3MD

Ventajas de JWT:

- Stateless: El servidor no necesita almacenar información de sesión.
- Escalable: Funciona bien en microservicios y arquitecturas distribuidas.
- Portable: Puede usarse entre diferentes dominios (CORS-friendly).

Desventajas de JWT:

- Difícil de invalidar antes de que expire (sin lista de tokens revocados).
- Si el payload crece, el tamano del token aumenta en cada peticion.
- Si se filtra la clave secreta, todos los tokens quedan comprometidos.

2.2 Sesiones (Session-based Auth)

La autenticación basada en sesiones es el enfoque tradicional utilizado por la mayoría de aplicaciones web clásicas. Cuando un usuario se autentica, el servidor crea una sesion y almacena su estado, devolviendo al cliente un identificador de sesion (Session ID) generalmente en una cookie.

Flujo de una sesión:

- **1.** El usuario envía credenciales al servidor.
- **2.** El servidor valida y crea una sesión en memoria o base de datos.
- **3.** Se envía al cliente un Session ID en una cookie segura.
- **4.** En cada petición, el cliente envía el Session ID y el servidor verifica la sesión.
- **5.** Al cerrar sesión, la sesión se elimina del servidor.

Ventajas: Fácil invalidación (solo borrar la sesión), historial centralizado, menor exposición de datos en el cliente. Desventajas: Requiere almacenamiento de estado en el servidor, puede ser un cuello de botella en sistemas de alta concurrencia.

2.3 OAuth 2.0

OAuth 2.0 es un framework de autorización (no autenticación) que permite a aplicaciones de terceros acceder a recursos en nombre de un usuario, sin que este deba compartir sus credenciales. Es el estándar utilizado por 'Iniciar sesión con Google/Facebook/GitHub'.

Roles en OAuth 2.0:

- Resource Owner: El usuario dueño de los datos.
- Client: La aplicación que solicita acceso (tu app).
- Authorization Server: El servidor que emite tokens (ej. Google).
- Resource Server: El API que protege los recursos (ej. Google Drive API).

Flujos principales de OAuth 2.0:

- Authorization Code Flow: El más seguro, usado en aplicaciones web con servidor backend.
- Implicit Flow: Obsoleto, era para SPAs sin backend (no recomendado en la actualidad).
- Client Credentials: Para comunicación servidor a servidor (sin usuario).
- PKCE (Proof Key for Code Exchange): Extensión del Authorization Code para apps móviles y SPAs.

Nota importante: OAuth 2.0 por si solo no proporciona autenticacion. Para eso existe OpenID Connect (OIDC), que es una capa de identidad construida sobre OAuth 2.0 que agrega un ID Token con información del usuario.

3. ¿Cuándo usar Tokens vs Sesiones?

La elección entre tokens (JWT) y sesiones depende de varios factores como la arquitectura del sistema, los requisitos de escalabilidad y las necesidades de seguridad.

Usar Tokens (JWT)	Usar Sesiones
APIs REST y microservicios	Aplicaciones web monolíticas tradicionales
Arquitecturas distribuidas o en la nube	Cuando se necesita invalidación inmediata

Apps móviles y Spas	Aplicaciones con datos sensibles en servidor
Múltiples dominios o servicios (SSO)	Sesiones de usuario de corta duración
Comunicación entre servicios (M2M)	Cuando el cliente es solo un navegador web
Cuando el servidor debe ser stateless	Cuando el control total de sesión es prioritario

Regla general: Si construyes una API consumida por multiples clientes (web, movil, IoT), prefiere JWT. Si construyes una aplicacion web traditional con un solo frontend, las sesiones son perfectamente validas y en muchos casos mas seguras.

4. Riesgos de Seguridad en un Mal Diseño de Autenticación

Un sistema de autenticación mal diseñado puede comprometer completamente la seguridad de una aplicación. Los siguientes son los riesgos más críticos identificados por OWASP y la industria de ciberseguridad.

4.1 Broken Authentication (OWASP A07:2021)

Ocurre cuando las funciones de autenticación son implementadas incorrectamente, permitiendo a atacantes comprometer contraseñas, claves de sesión o explotar otros fallos de implementación.

- Contraseñas débiles permitidas sin validación de fortaleza.
- Ausencia de protección contra ataques de fuerza bruta.
- Tokens de sesión predecibles o con baja entropía.
- URLs con Session IDs expuestos en logs o historial.

4.2 Vulnerabilidades Especificas de JWT

- Algoritmo 'none': Un JWT puede ser firmado con algoritmo 'none', omitiendo la verificacion de firma. El servidor NUNCA debe aceptar este algoritmo.
- Confusion RS256 vs HS256: Un atacante puede cambiar el header de RS256 a HS256 y firmar el token con la clave pública del servidor como si fuera una clave simétrica.
- Tokens sin expiración (sin campo 'exp'): Un token robado es válido indefinidamente.
- Almacenamiento inseguro: Guardar JWT en localStorage lo expone a ataques XSS.

4.3 Ataques Comunes

Ataque	Descripcion	Mitigacion
Credential Stuffing	Uso de credenciales robadas de otras brechas	MFA, detección de IPs anómalas
Session Hijacking	Robo del Session ID via XSS o red insegura	HTTPS, HttpOnly, Secure cookies
CSRF	Acciones no autorizadas con sesión activa del usuario	CSRF tokens, SameSite cookies
Brute Force	Intentos masivos de adivinar contraseñas	Rate limiting, bloqueo de cuenta
Token Replay	Reutilizar un token interceptado	Tokens de corta duración, nonce
JWT Tampering	Modificar el payload del token	Verificar firma siempre, usar RS256

4.4 Buenas Prácticas de Seguridad

- Siempre usar HTTPS para transmitir credenciales y tokens.
- Implementar MFA (Autenticación Multifactorial) especialmente para cuentas administrativas.
- Usar bcrypt, Argon2 o scrypt para almacenar contraseñas con salt.
- Configurar tiempos de expiración adecuados en tokens y sesiones.
- Implementar refresh tokens con rotación para extender sesiones de forma segura.
- Almacenar JWT en cookies HttpOnly con flag Secure en lugar de localStorage.
- Implementar rate limiting y bloqueo temporal después de múltiples intentos fallidos.

5. Roles y Permisos en Sistemas Reales

El control de acceso basado en roles (RBAC) y sus variantes son el estándar de la industria para gestionar quienes pueden hacer que dentro de una aplicación.

5.1 Modelos de Control de Acceso

RBAC - Role-Based Access Control

El modelo más común. Los permisos se asignan a roles, y los roles se asignan a usuarios. Un usuario puede tener múltiples roles.

- Ejemplo: rol 'admin' tiene permisos de lectura, escritura y eliminación.
- Ejemplo: rol 'editor' tiene permisos de lectura y escritura solamente.
- Ejemplo: rol 'viewer' solo tiene permiso de lectura.

ABAC - Attribute-Based Access Control

Control de acceso basado en atributos del usuario, del recurso y del entorno. Mas flexible y granular que RBAC pero más complejo de implementar.

- Ejemplo: 'Un médico puede ver el expediente de un paciente si pertenece al mismo hospital'.
- Combina multiples atributos: rol del usuario, departamento, horario, ubicacion.

ACL - Access Control List

Lista de control de acceso que especifica permisos para cada recurso individualmente. Muy granular, pero difícil de mantener a escala.

- Ejemplo clásico: permisos de archivos en sistemas Unix (rwxrwxrwx).
- Usado en servicios cloud como AWS S3 para permisos de objetos especificos.

5.2 Diseño de Roles en Sistemas Reales

Ejemplo de estructura de roles en una aplicación empresarial:

Rol	Permisos	Ejemplo de uso
Super Admin	CRUD completo en todo el sistema	Equipo de infraestructura
Admin	CRUD en módulos asignados	Gerentes de área
Editor	Crear y editar contenido propio	Creadores de contenido
Moderador	Leer y moderar contenido de otros	Equipo de soporte
Usuario	Leer y gestionar su propia cuenta	Clientes finales

Invitado	Solo lectura de contenido publico	Visitantes no registrados

5.3 Implementación en Sistemas Modernos

En una API REST con Node.js/Express, la autorización basada en roles típicamente se implementa como middleware:

En plataformas como Firebase, los roles y permisos se definen mediante Custom Claims y Firestore Security Rules, permitiendo definir reglas granulares directamente en la base de datos.

5.4 Principios Fundamentales

- Principio de mínimo privilegio: Otorgar solo los permisos estrictamente necesarios para cada rol.
- Separación de responsabilidades: Un solo rol no debería tener control total sobre procesos críticos.
- Defensa en profundidad: Validar permisos tanto en el frontend como en el backend.
- Auditoria: Registrar todas las acciones que realizan usuarios con roles privilegiados.
- Revisión periódica: Los roles y permisos deben revisarse regularmente para eliminar accesos innecesarios.